
Captioning sensoriale del formaggio Trentingrana

Dalla costruzione del dataset al confronto di tre metodi
encoder-decoder

Progetto **AI4FQC** — Project 07: *GRANA Captioning*

Relazione tecnica esaustiva

Dominio	Analisi sensoriale di sezioni di forme di grana
Acquisizione	Analizzatore visivo elettronico IRIS
Attributi	7 dimensioni sensoriali
Dataset finale	38.437 coppie immagine-caption, 1.497 immagini
Modelli	m1 (CNN+LSTM), m3 (ViT+Transformer), m6 (ViT+GePpeTto)
Costo LLM totale	~ \$5,60 (Anthropic Batch API, Haiku 4.5)

Sommario

Questa relazione documenta in modo completo il lavoro svolto nel progetto di *captioning sensoriale* del formaggio Trentingrana, articolato nei due passi richiesti dalla traccia AI4FQC: (1) la pulizia e pre-elaborazione delle descrizioni testuali dei degustatori, e (2) l'applicazione e il confronto di tre metodi di captioning encoder–decoder concettualmente diversi.

Coerentemente con il peso effettivo del lavoro, circa il 70% della relazione è dedicato alla **costruzione del dataset** — la parte più complessa e originale del progetto — e il restante 30% alla **selezione dei modelli e all'addestramento**. La pipeline di preparazione dati trasforma 51.988 righe grezze, eterogenee e dialettali in 38.437 coppie immagine–caption pulite, normalizzate e in forma di frase italiana, con un costo LLM contenuto a ~ \$5,60 grazie a una strategia deterministica-prima-dell'LLM. Sul fronte modellistico, il risultato più informativo emerge dallo *shuffle test*: l'encoder visivo — e non il decoder — è il fattore determinante per stabilire se il modello usi davvero l'immagine.

Indice

Sommario 1

1 Introduzione e contesto 2

1.1 I due passi della traccia 2

1.2 Struttura della relazione 2

Parte I — Costruzione del dataset 2

2 I dati grezzi e il problema del join 2

2.1 Il disallineamento centrale 3

2.2 Decisione: broadcast a livello di caseificio 3

2.3 Anatomia di una difficoltà invisibile 3

3 Architettura della pipeline 4

4 Fase 1 — Preparazione deterministica 4

5 Fasi 2–3 — Vocabolario controllato e audit 5

5.1 Perché un lemmatizzatore custom 6

6 Fase 4 — Pulizia e qualitizzazione delle misure 6

6.1 Da numeri a qualità 6

6.2 Deduplicazione 7

7 Fase 5 — Drop conservativo del rumore 7

8 Fasi 6–8 — Riscrittura tramite LLM 8

8.1 Design del prompt (Fase 6) 8

8.2 Pilot run (Fase 7) 8

8.3 Batch completo (Fase 8) 8

8.4 Validazione programmatica dell'output 9

9 Fase 9 — Salvage manuale	9
10 Fase 10 — Broadcast finale e forma di frase	10
10.1 Forma di frase (sentence form)	10
11 Il dataset finale	10
11.1 Schema del dataset consegnato	11
11.2 Decisioni chiave e tradeoff	11
Parte II — Selezione dei modelli e training	12
12 I tre metodi e gli assi di variazione	12
13 Setup sperimentale	12
13.1 Salute dell’addestramento	13
14 Risultati: per-attributo e globale	13
14.1 Quadro sintetico per attributo	13
14.2 Perché i numeri per-attributo sono molto più alti	14
15 Lo shuffle test: il modello usa l’immagine?	14
15.1 La conclusione architetturale	15
16 Oltre BLEU: metriche complementari	16
16.1 CLIPScore conferma lo shuffle test	17
16.2 CIDEr discrimina dove BLEU appiattisce	18
16.3 Il tranello di BLEU, visualizzato	18
17 Esempi qualitativi	18
18 Critica di BLEU e conclusioni	19
18.1 Perché BLEU è un cattivo giudice qui	19
18.2 Conclusione generale	20
18.3 Framing onesto	20
18.4 Lavori futuri	20

1 Introduzione e contesto

Il progetto nasce all'interno dell'iniziativa **AI4FQC** (*Artificial Intelligence for Food Quality Control*) e ha per oggetto il *Project 07 – GRANA Captioning*. L'obiettivo è generare automaticamente descrizioni sensoriali, in lingua italiana, a partire da immagini di sezioni di forme di formaggio grana acquisite con l'analizzatore visivo elettronico **IRIS** in condizioni di illuminazione controllata.

Ogni campione di formaggio è corredato da descrizioni redatte da un *panel* di degustatori esperti, che coprono **sette attributi sensoriali**:

■ Profumo	■ Aroma	■ Sapore	■ Texture
■ Spessore della Crosta	■ Struttura della Pasta	■ Colore della Pasta	

1.1 I due passi della traccia

La traccia richiede esplicitamente due deliverable:

1. **Passo 1 — Pre-elaborazione del testo.** Pulire e normalizzare le descrizioni telegrafiche, dialettali e incoerenti dei degustatori, sostituendo in particolare le descrizioni quantitative (misure in mm/cm) con descrizioni qualitative.
2. **Passo 2 — Tre metodi di captioning.** Applicare e confrontare tre metodi encoder-decoder “concettualmente quanto più diversi possibile”. L'enfasi è sul *confronto di metodi*, non sulla ricerca di un singolo modello “migliore”.

1.2 Struttura della relazione

La Parte I (sezioni 2–11) ripercorre fase per fase la costruzione del dataset, dalla tabella grezza al file finale, motivando ogni scelta tecnica. La Parte II (sezioni 12–16) descrive la scelta dei tre modelli, il setup di addestramento su Kaggle, i risultati per-attributo e globali, e l'analisi di *image-conditioning*. Tutti i numeri riportati sono stati verificati direttamente contro il codice e i file di dati del repository.

In una riga

2.745 fotografie + 13.159 commenti grezzi → pipeline di 11 fasi → 38.437 coppie immagine-caption pulite → 3 metodi encoder-decoder confrontati su 7 attributi.

Parte I — Costruzione del dataset

2 I dati grezzi e il problema del join

Il punto di partenza era ingannevolmente ricco ma profondamente disallineato:

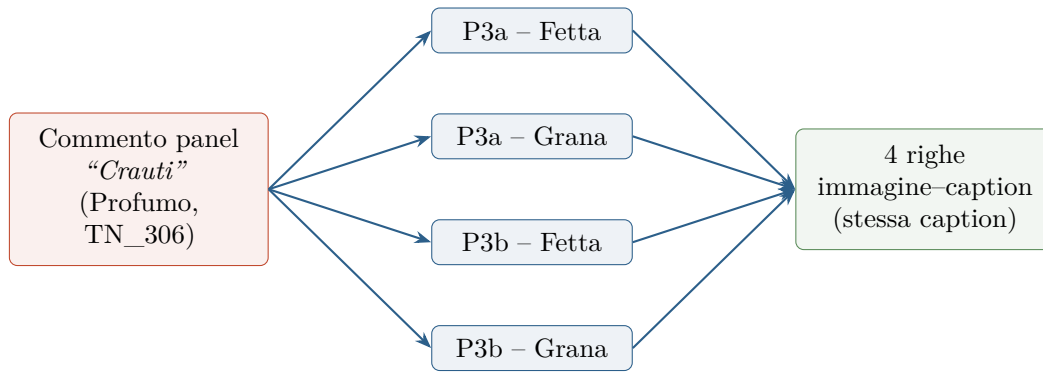


Figura 1: Join per broadcast a livello di caseificio. In assenza di un join a livello di forma, ogni commento è replicato su tutte le immagini coerenti. Più righe per immagine sono un vantaggio per l’addestramento (più dati), non un difetto.

- **2.745 fotografie BMP** (1024×768 , RGB) di sezioni di forme, organizzate per anno e per seduta. Ogni nome file codifica lo slot del panel, l’identificativo del caseificio e la vista: **Fetta** (fetta della forma) o **Grana** (primo piano della grana). Esempio: `P3a_TN306_612_Fetta.bmp`.
- **Quattro workbook Excel** (uno per anno: 2018–2021), con un foglio per attributo sensoriale. Ogni riga è il testo libero di un degustatore su una forma per un attributo, più eventuali punteggi numerici.
- **Un codebook** che mappa identificativi caseificio $\leftrightarrow$ codici prodotto $\leftrightarrow$ lettere (16 caseifici).

2.1 Il disallineamento centrale

Il problema non ovvio era che *immagini e commenti parlavano lingue diverse*: i nomi file identificavano la singola forma (es. 612 è un ID campione), mentre i commenti identificavano solo il *vassoio* del degustatore (**Prodotto** = COD). Non esisteva un join a livello di riga: solo un join a livello di *caseificio*, attraverso il codebook.

2.2 Decisione: broadcast a livello di caseificio

Ogni commento del degustatore è stato propagato (*broadcast*) a *tutte* le fotografie di quel caseificio in quella seduta. Un singolo commento “Crauti” per il Profumo di un caseificio diventa così la caption di più immagini (repliche $a/b \times$ viste **Fetta/Grana**), fino a 4 righe immagine-caption da un solo commento. La Figura 1 illustra il meccanismo.

L’effetto netto del join: **51.988 righe** (immagine $\times$ panelista $\times$ attributo) su 2.745 immagini, di cui **39.510** con commento non vuoto (tasso di pairing $\sim 76\%$, più basso nelle sedute 2021 dove alcuni campi attributo erano lasciati vuoti).

2.3 Anatomia di una difficoltà invisibile

Tre ostacoli concreti, ognuno costato tempo reale, hanno reso il join non banale:

- **Tripla indicizzazione del caseificio.** Lo stesso caseificio appare nei dati come TN_306, TN306 e 306; la mappa di join li normalizza tutti alla stessa chiave per non perdere righe.

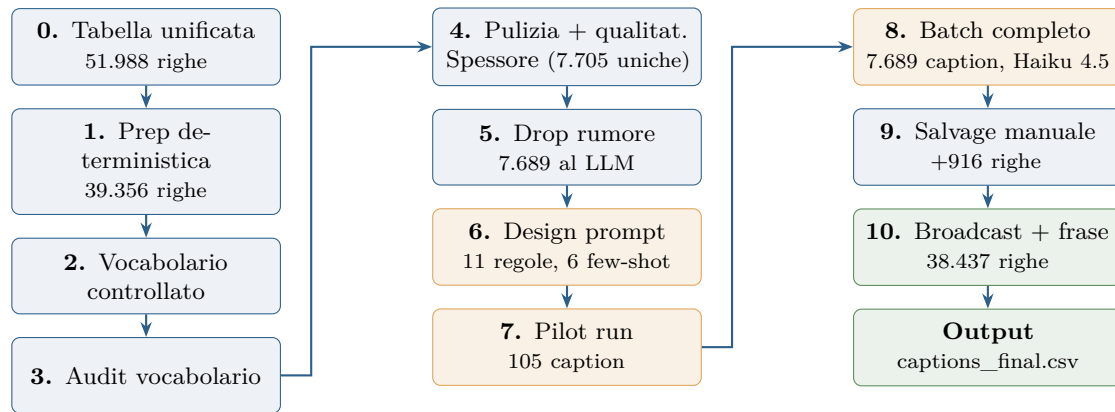


Figura 2: Le 11 fasi della pipeline. In **blu** le fasi deterministiche (codice, gratuite), in **oro** le fasi che coinvolgono l’LLM, in **verde** l’assemblaggio finale.

- **Incoerenze di header tra anni.** “Spessore della Crosta” vs “Spessore della crosta”, “N° Seduta” vs “Seduta”: normalizzati prima del merge.
- **Virgola decimale italiana.** I punteggi numerici 2018 (‘7,48’) richiedono parsing localizzato a `float`.

Tutti i metadati di provenienza (panelista, data seduta, caseificio, slot, replica, vista) vengono propagati a valle, così ogni riga finale resta tracciabile fino al commento originale.

3 Architettura della pipeline

La pipeline è organizzata in **11 fasi** (numerate 0–10), ognuna con un proprio script Python, input/output espliciti e report ispezionabile. Il principio guida è *deterministico-prima-dell’LLM*: tutto ciò che può essere fatto con codice riproducibile e gratuito viene fatto prima di interpellare il modello linguistico. La Figura 2 ne mostra il flusso complessivo.

La Figura 3 quantifica come il numero di righe si riduce lungo la pipeline a livello di *broadcast* (righe di addestramento), mentre la Figura 4 mostra la riduzione sul piano delle *caption uniche*, che è ciò che determina il costo dell’LLM.

4 Fase 1 — Preparazione deterministica

La prima fase filtra e normalizza il testo *senza riformularlo*. Le operazioni principali:

- **Filtro righe vuote/N/A** e normalizzazione Unicode (NFC), rimozione di spazi indivisibili (`\xa0`), caratteri a larghezza zero, tabulazioni e ritorni a capo.
- **Drop di meta-commenti** via una piccola blacklist conservativa (varianti di N/A, “non penalizzo”, “non valuto”, righe di sola punteggiatura).
- **Drop di rumore quasi-vuoto:** meno di 2 caratteri alfanumerici dopo la pulizia.
- **Doppia colonna** `caption_raw` / `caption_norm` per garantire reversibilità completa e auditabilità.

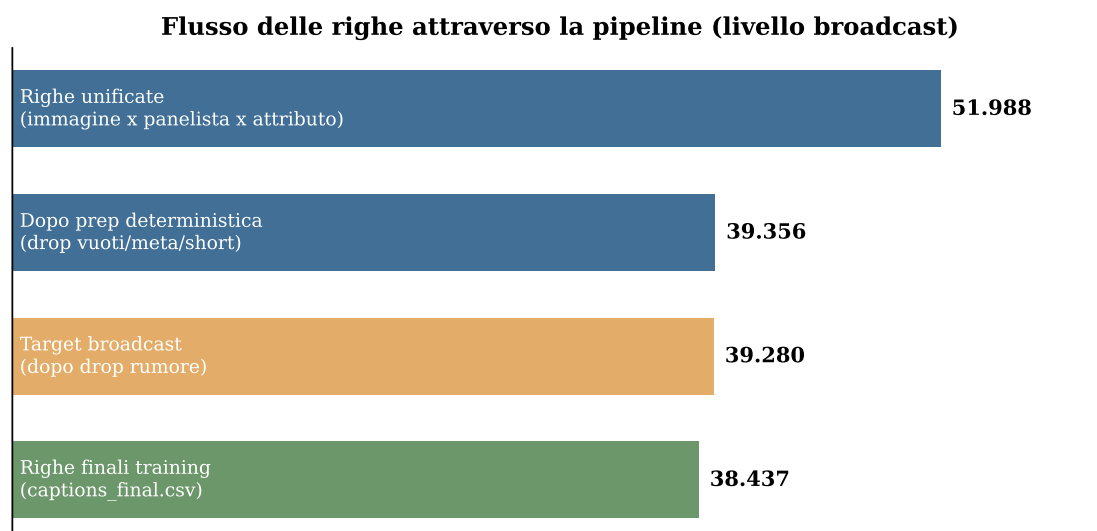


Figura 3: Flusso delle righe di addestramento attraverso la pipeline. Lo scarto maggiore (12.632 righe) avviene nella prep deterministica, eliminando vuoti, meta-commenti e frammenti senza contenuto.

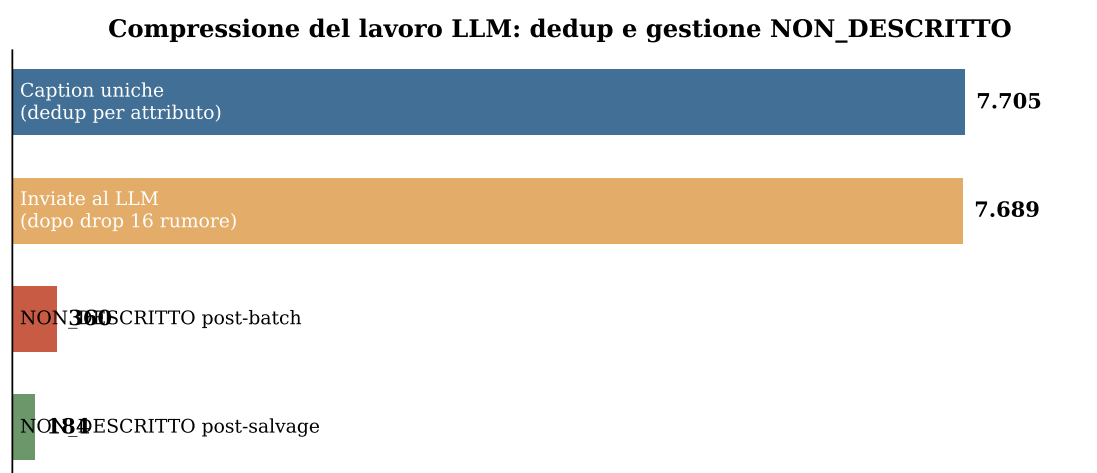


Figura 4: Compressione del lavoro LLM. La deduplicazione riduce ~39k righe a 7.705 caption uniche (compressione 5,1×); solo queste vengono inviate al modello.

L’output è di **39.356 righe** (75,7% di retention), con **12.632 drop** così ripartiti: 12.478 vuoti/null, 86 meta-commenti, 68 quasi-vuoti. Una scelta chiave è stata mantenere una blacklist *piccola*: a questo stadio i drop devono essere *certi*, perché l’LLM gestirà meglio le ambiguità (es. “non penalizzo, ma sa di stalla” contiene un descrittore reale e non va scartato).

5 Fasi 2–3 — Vocabolario controllato e audit

Per ciascuno dei 7 attributi viene estratto un **vocabolario controllato** dei lemmi e dei bigrammi più frequenti, con una lemmatizzazione italiana *custom* (mappe deterministiche di abbreviazioni e refusi, ripristino degli accenti finali, dizionario SPECIAL per i casi ambigui, e un merge corpus-driven singolare↔plurale che scatta solo quando entrambe le forme sono attestate).

5.1 Perché un lemmatizzatore custom

Le librerie NLP italiane standard sbagliano sistematicamente sul lessico sensoriale (*panna*→*panno*, *latte*→*latto*). Inoltre includono nelle stopwords parole che qui *sono* informative (*molto*, *poco*, *leggermente*, intensificatori sensoriali). Il vocabolario serve a due scopi: (1) come base per l’audit, e (2) come *ancora stilistica* nel prompt LLM — non come dizionario chiuso. Poiché il modello parla già italiano fluentemente, non serve un lemmatizzatore perfetto: basta che la forma canonica sia attestata.

La Figura 5 mostra l’ampiezza del vocabolario per attributo. *Struttura della Pasta* è il più ricco (777 lemmi, 41.676 token), *Spessore della Crosta* il più povero (252 lemmi): coerente con la loro diversità descrittiva intrinseca.

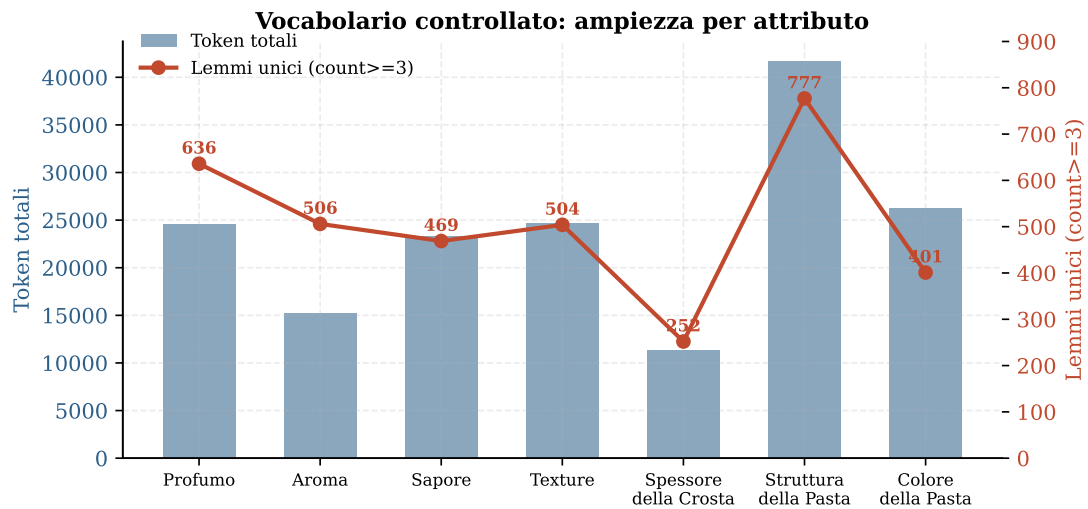


Figura 5: Vocabolario controllato per attributo: barre = token totali (asse sinistro), linea = lemmi unici con frequenza ≥ 3 (asse destro). La fase di audit (`audit_vocabulary.py`) ha segnalato token quantitativi, stem spezzati e refusi tramite distanza di Levenshtein, in tre round di correzioni.

6 Fase 4 — Pulizia e qualitizzazione delle misure

Questa fase deterministica esegue tre operazioni: pulizia testuale parola per parola (espansione di abbreviazioni e refusi preservando il *casing*), **qualitizzazione delle misure di crosta**, e **deduplicazione**.

6.1 Da numeri a qualità

La traccia chiede esplicitamente di sostituire le descrizioni quantitative con qualitative. Per lo Spessore della Crosta, centinaia di caption erano semplici numeri (10, 1 cm, Mediamente 9 mm, 8–10 mm). Una funzione deterministica riconosce le caption *interamente* numeriche (con/senza unità, range, prefissi-qualificatori), converte tutto in millimetri (con euristica $< 5 \rightarrow \text{cm}$) e assegna un bucket qualitativo:

Soglia (mm)	Bucket
< 8	Molto sottile
$8 \leq x < 10$	Sottile
$10 \leq x < 14$	Media
$14 \leq x < 18$	Spessa
≥ 18	Molto spessa

Questo **elimina alla radice** la principale fonte di incoerenza dell’LLM, che bucketizzava 1 cm e 10 mm (stessa misura fisica) in modo diverso. La Figura 6 mostra la distribuzione risultante: 424 righe numeriche collassate in 5 bucket.

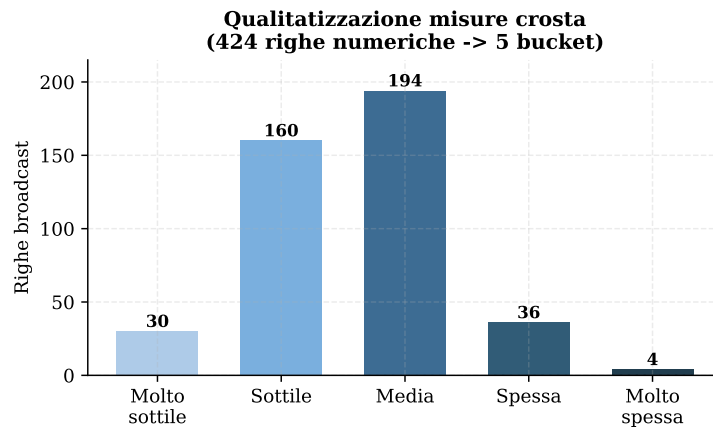


Figura 6: Qualitizzazione delle misure di crosta. Le caption interamente numeriche sono mappate deterministicamente sui 5 bucket; le caption miste (es. “spigoli sopra 20mm, piatto 10mm”) sono lasciate all’LLM perché richiedono ragionamento contestuale.

6.2 Deduplicazione

A causa del broadcast, lo stesso testo sarebbe stato inviato all’LLM molte volte. La deduplicazione per chiave (caption, attributo) comprime 39.356 righe in **7.705 caption uniche** (compressione 5,1×). La Figura 7 mostra la compressione per attributo. Questo è il singolo accorgimento che riduce di $\sim 5\times$ il costo dell’LLM.

7 Fase 5 — Drop conservativo del rumore

Uno script di *survey* classifica le caption uniche in 7 categorie (meta, valutazioni pure, intensificatori puri, incertezza, interrogativi, soli numeri, vuoti post-tokenizzazione) senza droppe, producendo un report ispezionabile. Solo successivamente uno script di *drop attivo* elimina le 3 categorie *certe* (valutazioni pure come “Buono”/“Ok”, soli numeri fuori-Spessore, meta di sistema).

Il drop è deliberatamente conservativo: **16 caption uniche / 76 righe broadcast (0,2%)**. Le caption che mescolano meta e descrittore vengono *mantenute* — l’LLM farà quella “chirurgia” meglio di una regex. Vengono inoltre preservati gli interrogativi (“Eucalipto?” → poi trasformato in affermazione) e le negazioni descrittive (“Non paglierino”), perché portano informazione sensoriale reale. Restano **7.689 caption uniche** pronte per il modello.

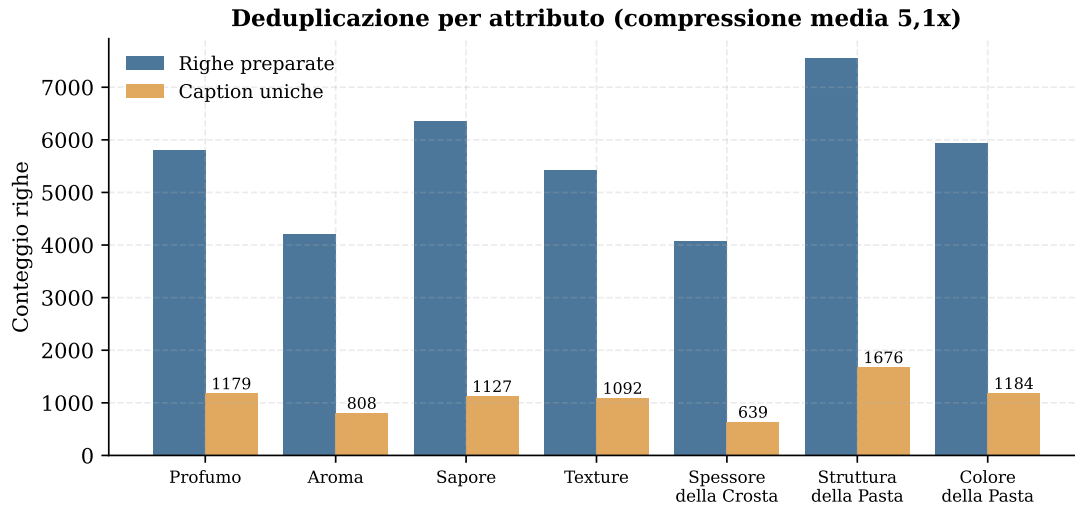


Figura 7: Righe preparate vs caption uniche per attributo. Il pattern dominante di broadcast è $4\times$ (repliche $a/b \times$ viste **Fetta/Grana**): 6.595 caption compaiono esattamente 4 volte.

8 Fasi 6–8 — Riscrittura tramite LLM

L’LLM svolge il lavoro genuinamente difficile che nessuna regex potrebbe fare fedelmente: espandere parole telegrafiche in caption naturali (“Crauti” → “Profumo di crauti.”), normalizzare il dialetto, rimuovere i meta-commenti preservando le parti descrittive, convertire misure *incorporate* in descrizioni qualitative e trasformare le domande in affermazioni.

8.1 Design del prompt (Fase 6)

Un prompt di sistema per attributo (~ 5 KB ciascuno) combina: ruolo e framing del dataset, etichetta dell’attributo, template di stile, **11 regole** (tra cui la regola 6 di “zero invenzione” e la regola 11 dell’escape `NON_DESCRITTO`), regole extra per-attributo (tabella di conversione mm/cm per lo Spessore), i **top-60 lemmi** del vocabolario controllato come ancora stilistica, idiomi multi-parola curati a mano, e **6 esempi few-shot** tratti da caption reali.

8.2 Pilot run (Fase 7)

Un campione stratificato di 105 caption (15 per attributo) è passato per primo attraverso Haiku 4.5 in modalità sincrona. Il pilot ha rivelato due modi di fallimento, entrambi corretti nel prompt: (1) l’incoerenza 1 cm vs 10 mm, risolta con la tabella di conversione esplicita; (2) violazioni di formato sugli input senza contenuto, risolte con l’escape `NON_DESCRITTO`. Lezione operativa: 3 worker in parallelo erano l’ottimo, 8 worker scatenavano tempeste di rate-limit (429).

8.3 Batch completo (Fase 8)

Tutte le 7.689 caption uniche sono state inviate come un singolo job Anthropic Batch API: **7.689/7.689 completate, 0 errori**, ~ 25 –30 minuti, $\sim \$4,50$. La Figura 8 motiva la scelta di Haiku 4.5: per questo compito di riscrittura ben delimitato, verificato sul pilot, modelli più costosi non offrivano alcun guadagno di qualità misurabile.

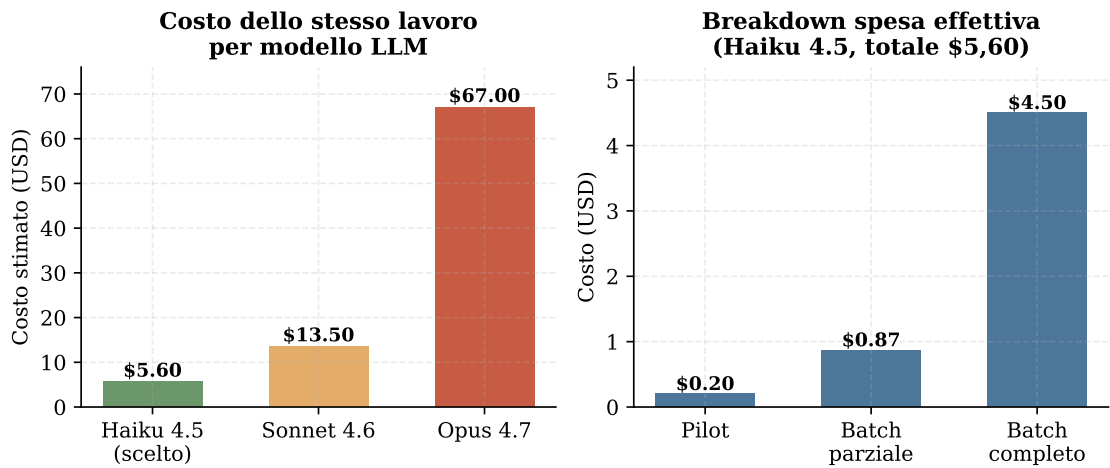


Figura 8: A sinistra: costo dello stesso lavoro con modelli diversi — Haiku è ~2,4× più economico di Sonnet e ~12× di Opus. A destra: ripartizione effettiva della spesa con Haiku 4.5 (totale \$5,60, pilot + batch parziale + batch completo).

8.4 Validazione programmatica dell’output

Un controllo automatico su tutte le 7.689 uscite ha verificato la conformità al Passo 1 della traccia. Il risultato è netto:

Controllo	Violazioni
Output inizia col prefisso d’attributo atteso	1 (prefisso alternativo valido)
Output contiene cifre	0
Output contiene unità (mm/cm/%)	0
Output più lungo di 25 parole	0
Output vuoto	0
Output multi-riga o con markup	0

La conversione quantitativo→qualitativo è riuscita al 100%, una soddisfazione pulita del requisito principale del Passo 1: *zero numeri e zero unità di misura residue* nelle caption finali.

9 Fase 9 — Salvage manuale

Una scansione euristica ha segnalato che 291 delle ~362 caption taggate NON_DESCRITTO contenevano almeno un lemma del vocabolario controllato — segno che l’LLM era stato *eccessivamente cauto* con la regola 11 su input borderline (giudizio + descrittore). È stata curata a mano una mappa di salvataggio di **178 caption** dove il descrittore era reale e fedele:

Sorgente	Salvataggio
"marcio, putrido,"	“Profumo marcio e putrido.”
"Strano. A tratti sentiva di pesce."	“Profumo strano, di pesce a tratti.”
"Sangue,, "	“Aroma di sangue.”
"Anonimo"	“Aroma anonimo.”

L’effetto netto (Figura 9): i NON_DESCRITTO unici scendono da 362 a **184**, recuperando **916 righe di addestramento**. Le caption rimaste NON_DESCRITTO erano genuinamente prive di

informazione (giudizi puri, frammenti incomprensibili, meta di sistema, osservazioni del tutto fuori-attributo).

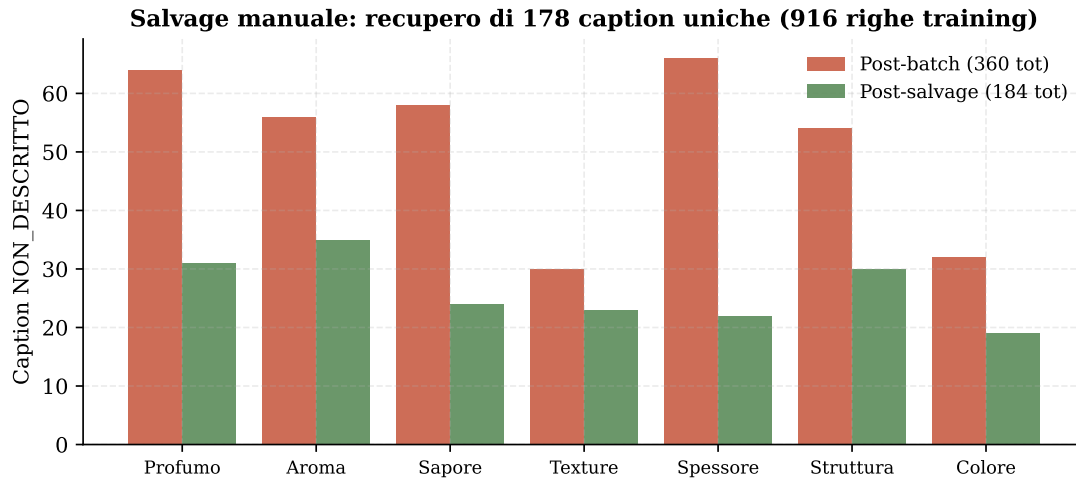


Figura 9: NON_DESCRITTO per attributo, prima (post-batch, 360) e dopo (post-salvage, 184) il salvataggio manuale. “Pulire i dati” non significa “buttarli via”: la cura manuale ha recuperato 916 righe altrimenti perse.

10 Fase 10 — Broadcast finale e forma di frase

Le 7.505 caption pulite (escluse le NON_DESCRITTO) vengono riconnesse alla tabella broadcast di 39.280 righe tramite la chiave di deduplicazione; le righe la cui caption pulita era NON_DESCRITTO vengono eliminate. Risultato: **38.437 righe** di addestramento su **1.497 immagini uniche**.

10.1 Forma di frase (sentence form)

Un requisito tardivo: fornire, accanto alla forma compatta (**Profumo di panna.**), una **frase italiana dichiarativa completa** (**Il formaggio ha un profumo di panna.**), più adatta agli encoder-decoder e alle metriche di captioning (BLEU, METEOR). La trasformazione è puramente deterministica (nessun LLM): canonicalizzazione dei prefissi, sostituzione di template, e un *polish pass* che corregge le imperfezioni grammaticali (iniezione dell’articolo dopo “presenta”, consapevolezza dei plurali, elisione *una*→*un’*). Copertura template del 100%, zero round-trip LLM aggiuntivi.

Entrambe le forme sono presenti in ogni riga; **tutto l’addestramento del Passo 2 usa la forma di frase** (`caption_sentence`) perché grammaticale, più informativa e più rappresentativa di come un umano descriverebbe il formaggio.

11 Il dataset finale

La Figura 10 riassume la distribuzione finale delle righe per attributo. Vi è una varianza significativa (Struttura ha $\sim 1,9\times$ le righe di Spessore), fatto che inciderà sul confronto cross-attributo nella Parte II.

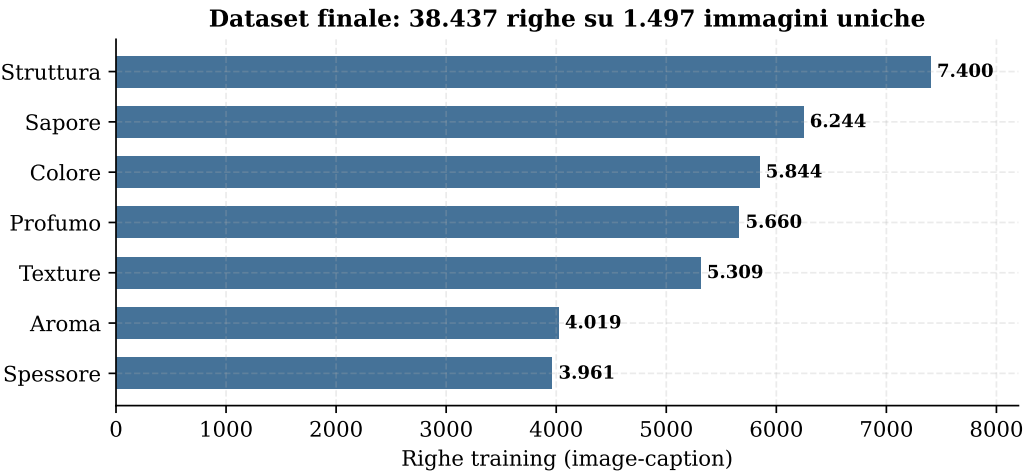


Figura 10: Righe di addestramento per attributo nel dataset finale (38.437 totali). Ogni immagine contribuisce con una riga per attributo coperto.

11.1 Schema del dataset consegnato

Il deliverable del Passo 1 è fornito in più forme, per servire qualsiasi architettura a valle:

File	Contenuto
captions_final.csv	tabella completa, 38.437 righe × 18 colonne (provenienza inclusa)
image_caption_attribute.csv	vista semplificata a 4 colonne: image_path, attribute, caption, caption_sentence
by_attribute/<Attr>.csv	7 split per-attributo, per chi addestra un modello per attributo
README.md	spiegazione del deliverable per gli utenti a valle

Ogni riga porta **due forme di caption**: `caption` (compatta, ~4–8 parole, ancorata all’attributo) e `caption_sentence` (frase italiana completa, ~7–15 parole). Sono inoltre conservate tutte le colonne di provenienza (panelista, data seduta, anno, vista, slot, replica, caseificio, codice prodotto), così ogni coppia immagine–caption resta tracciabile fino al commento originale del degustatore.

11.2 Decisioni chiave e tradeoff

La Tabella seguente raccoglie le scelte progettuali più rilevanti della Parte I, con la relativa motivazione — la traccia chiede una pulizia “in modo accurato”, e ogni decisione è stata presa per essere auditabile e giustificabile.

Decisione	Cosa	Perché
Granularità del join	Livello caseificio (broadcast a tutte le forme)	I dati non supportano un join a livello forma; più righe per immagine sono un vantaggio per il training
Deterministico prima dell’LLM	Pre-elaborazione pesante (typo, abbrev, qualitativizzazione, dedup)	Auditabile, gratuito, riduce il costo LLM di $\sim 5\times$
Vocabolario come ancora	Top-N lemmi come “preferisci”, non “usa solo”	L’LLM parla italiano; non serve un lemmatizzatore perfetto
Escape NON_DESCRITTO	Token singolo invece di rifiuto multi-riga	Segnale banale da filtrare a posteriori
Salvage manuale	Cura a mano di 178 caption borderline	Recupera 916 righe; più economico di un altro giro LLM
Doppia forma caption	Compatta + frase nella stessa riga	A valle si sceglie quella adatta all’architettura
Haiku 4.5 (non Sonnet/Opus)	$\sim \$5,60$ vs $\sim \$13,50$ vs $\sim \$67$	Il pilot ha provato che il modello piccolo gestisce il task con pari qualità

Riepilogo Parte I

Da 51.988 righe grezze e dialettali a 38.437 coppie immagine-caption pulite, normalizzate e in forma di frase. Strategia chiave: deterministico-prima-dell’LLM (dedup $5,1\times$, qualitativizzazione misure, drop conservativo) per ridurre il costo LLM a $\$5,60$; salvataggio manuale per recuperare 916 righe; doppia forma compatta/frase per la massima flessibilità a valle.

Parte II — Selezione dei modelli e training

12 I tre metodi e gli assi di variazione

La traccia chiede tre metodi encoder-decoder “concettualmente quanto più diversi possibile”. Sono state scelte tre architetture che isolano i tre assi principali di variazione nel captioning di immagini (Figura 11):

Tenere gli encoder congelati rende il confronto equo (stesse feature visive disponibili a tutti i decoder) e mantiene il budget sperimentale gestibile. La codebase implementa anche m2/m4/m5 per completare la griglia 2×3 , ma non sono stati eseguiti perché non aggiungono un *nuovo asse concettuale*.

13 Setup sperimentale

L’addestramento è avvenuto interamente sul tier GPU gratuito di **Kaggle** (T4/P100). La Tabella seguente riassume gli iperparametri (default ragionevoli per ciascuna architettura, non frutto di

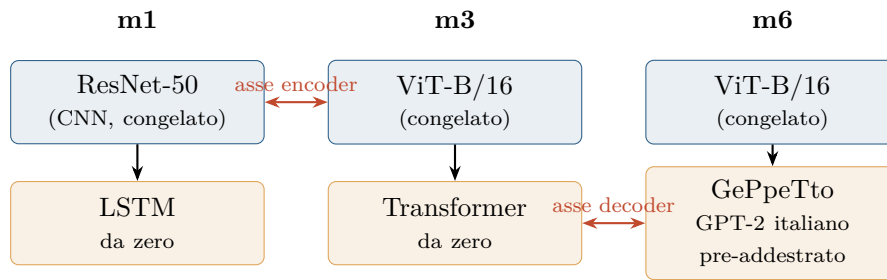


Figura 11: I tre metodi e gli assi che isolano. **m1 vs m3** isola l’*encoder* (CNN vs ViT, stesso decoder da-zero); **m3 vs m6** isola il *decoder* (stesso ViT, decoder da-zero vs LM italiano pre-addestrato). Tutti gli encoder sono *congelati*: solo il decoder e la proiezione di cross-attention vengono addestrati.

una ricerca):

Modello	epoche	batch	lr	scheduler	patience
m1 (CNN+LSTM)	50	32	3×10^{-4}	StepLR	7
m3 (ViT+Transf.)	30	16	1×10^{-4}	cosine	5
m6 (ViT+GePpeTto)	20	8	5×10^{-5}	cosine	5

Lo split è *sample-disjoint* (train 674 / val 143 / test 147 campioni): lo stesso formaggio non compare mai sia in train sia in test, evitando il leak. In inferenza, tutti i modelli decodificano con **nucleus sampling** ($\text{top-}p = 0,9$, $T = 0,7$) invece di beam search: su dataset piccoli il beam collassa sulla caption modale, gonfiando il BLEU ma riducendo la diversità. Lezione Kaggle costosa: il flag `enable_gpu` nei metadati *non* abilita davvero la GPU (va attivata manualmente), e il tier gratuito tende a P100 (sm_60) la cui PyTorch preinstallata va reinstallata da wheel cu118.

13.1 Salute dell’addestramento

Prima dei risultati, vale la pena notare che le curve di addestramento sono sane su tutti e tre i modelli: nessun segnale di overfitting, nessun NaN, nessun collasso. m1 ha completato 50/50 epoche con `val_loss` in calo costante ($1,84 \rightarrow 1,04$); m3 si è fermato in early-stop all’epoca 16/30 (patience 5); m6 ha completato 20/20 con miglior `val_loss` 1,06 all’epoca 16. Le differenze nei risultati a valle non sono quindi attribuibili a patologie di training.

14 Risultati: per-attributo e globale

I modelli sono stati valutati in due regimi: **per-attributo** (7 modelli separati) e **globale** (un modello su tutti gli attributi insieme). La Figura 12 mostra il BLEU-4 per-attributo confrontato con la baseline `most_frequent` (che emette sempre la caption modale).

14.1 Quadro sintetico per attributo

La Tabella riassume, per ciascun attributo, la dimensione del test, il modello vincente per BLEU-4 e il margine sulla baseline costante. Emerge una netta separazione tra attributi “difficili” (caption diverse, i modelli vincono) e “apparentemente facili ma template-heavy” (poche caption modali, la baseline è dura da battere).

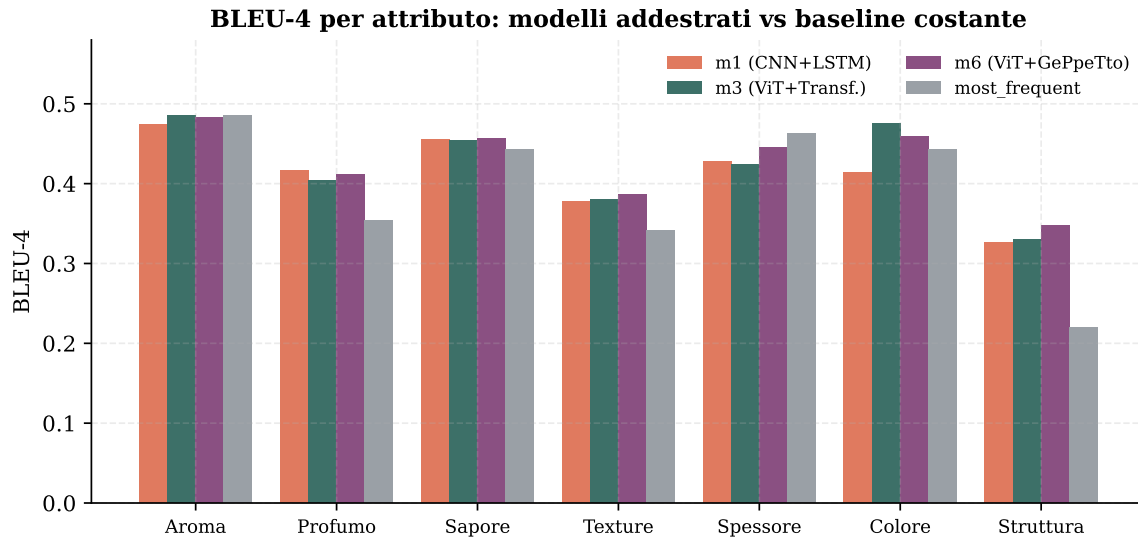


Figura 12: BLEU-4 per attributo. I modelli addestrati battono nettamente la baseline costante su Struttura (+0,128), Profumo, Texture e Colore; su Aroma, Sapore e Spessore pareggiano o perdono — non perché siano scarsi, ma perché BLEU premia il predittore modale su distribuzioni di caption a bassa diversità.

Attributo	Test N	Migliore	Δ vs most_freq	Tipo
Struttura della Pasta	490	m6	+0,128	difficile
Profumo	405	m1	+0,062	difficile
Texture	394	m6	+0,045	difficile
Colore della Pasta	421	m3	+0,033	difficile
Sapore	443	m6	+0,014	pareggio
Aroma	307	m3	−0,000	pareggio
Spessore della Crosta	291	m6	−0,018	baseline vince

I modelli addestrati battono la baseline su 4 attributi su 7. Dove pareggiano o perdono (Aroma, Sapore, Spessore) non è per debolezza: su Spessore lo shuffle test (Figura 15) conferma che m3 e m6 *usano* l'immagine ($z > 3$), ma BLEU premia comunque il predittore modale.

14.2 Perché i numeri per-attributo sono molto più alti

Il BLEU-4 globale di m1 è 0,128; quello per-attributo va da 0,33 a 0,47. Questo **non** significa che i modelli per-attributo siano migliori (Figura 14): la valutazione per-attributo gira su una distribuzione molto più stretta. Il vocabolario collassa a ~ 80 –140 parole invece di $\sim 600+$, e lo *scaffolding* condiviso (“Il formaggio ha un [attributo] di...”) contribuisce ~ 5 parole di 4-gram costante. Sono due righe diversi: misurano performance su distribuzioni diverse.

15 Lo shuffle test: il modello usa l'immagine?

L'analisi più informativa dell'intero progetto. Se un modello è davvero *condizionato dall'immagine*, allora la predizione per l'immagine i deve combaciare con il riferimento per i meglio di una predizione scelta a caso. Lo *shuffle test* mescola casualmente le predizioni fra le righe di test

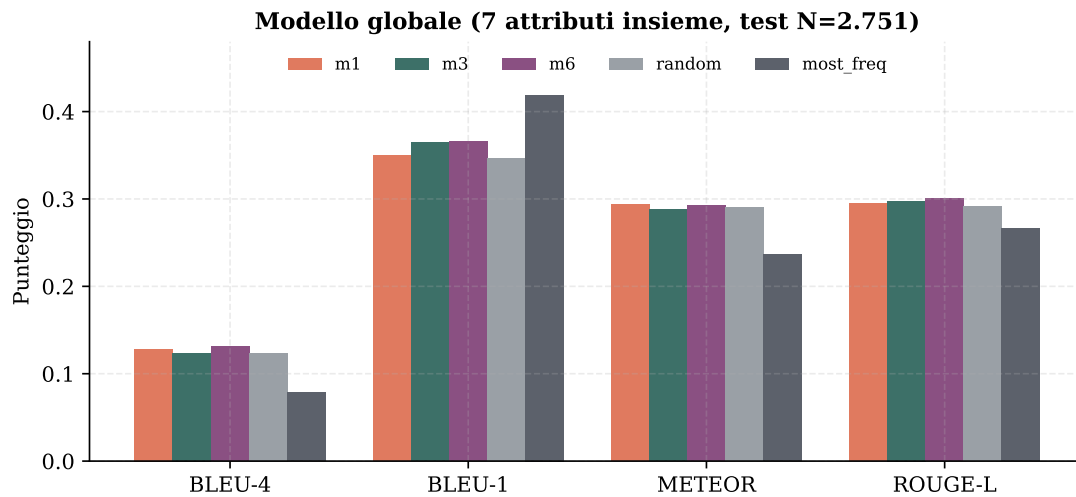


Figura 13: Modello globale (7 attributi insieme). m6 vince di misura su BLEU-4, BLEU-1 e ROUGE-L; i tre modelli restano in un cluster strettissimo (spread 0,007 BLEU-4). Nota: *most_frequent* ha il BLEU-1 più alto ma il BLEU-4 peggiore — comportamento degenerare “predici sempre la maggioranza”.

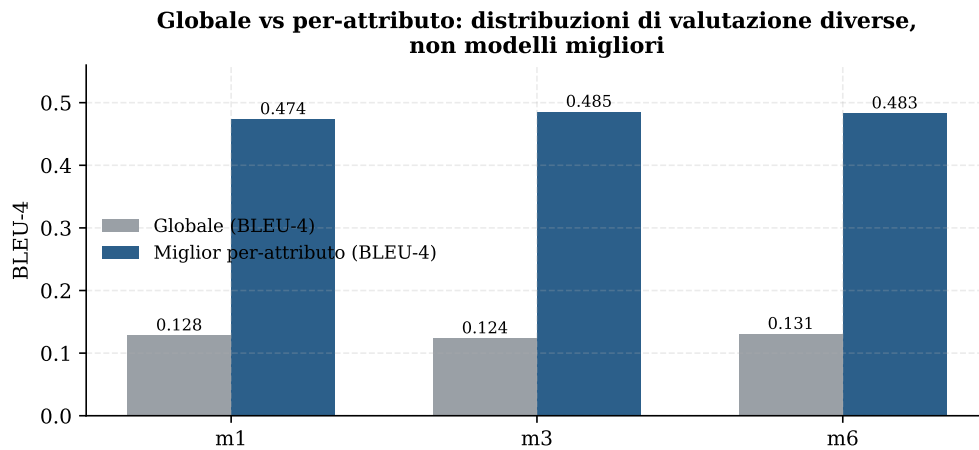


Figura 14: Globale vs miglior per-attributo (BLEU-4). Il salto 3–4× riflette il restringimento della distribuzione di valutazione, non un apprendimento migliore. Il modello globale è intrinsecamente più difficile: deve anche *scegliere quale* dimensione sensoriale descrivere e usa un vocabolario 7× più ampio.

(rompendo l’allineamento predizione↔immagine), ricalcola la sovrapposizione 100 volte per ottenere una distribuzione nulla, e calcola lo z-score. Uno $z > 3$ corrisponde a $p < 0,001$: forte evidenza di image-conditioning.

15.1 La conclusione architetturale

Il singolo determinante più importante dell’image-conditioning su questo dataset è l’**encoder visivo**, non il decoder, non la scala dei dati, non gli iperparametri:

- **m1 vs m3** (isola l’encoder): m1 non si condiziona mai all’immagine; m3 lo fa sulla maggior parte degli attributi.
- **m3 vs m6** (isola il decoder): producono output diversi (mai la stessa caption per la stessa immagine), ma hanno lo *stesso* profilo di image-conditioning — gli stessi attributi riescono,

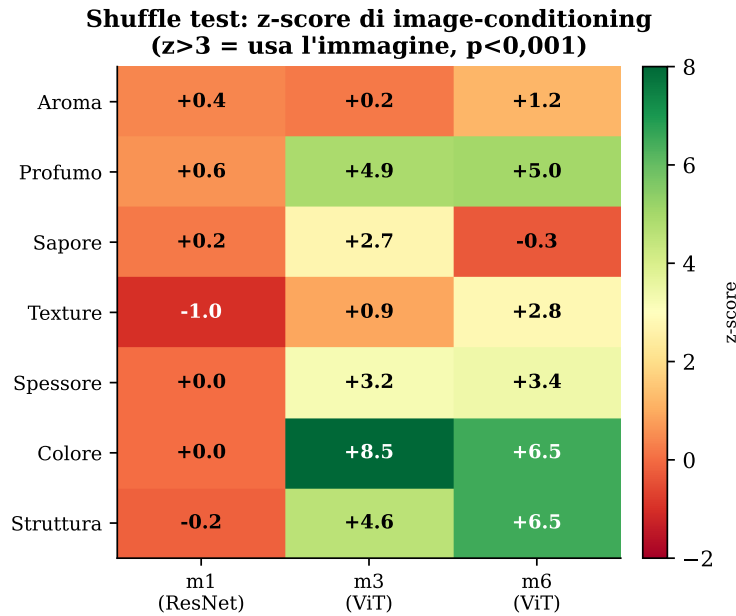


Figura 15: Shuffle test: z-score per modello e attributo. **m1 (ResNet)** non usa mai l'immagine ($z \approx 0$ ovunque): è di fatto un puro modello linguistico. m3 e m6 (ViT) usano l'immagine chiaramente su 4 attributi su 7 (Profumo, Spessore, Colore, Struttura). Aroma è l'unico dove nessun modello usa l'immagine.

gli stessi falliscono.

In altre parole: le feature di un ResNet-50 congelato non sono abbastanza informative a questa scala di dati, mentre quelle di un ViT-B/16 congelato sì, indipendentemente dal decoder posto a valle.

16 Oltre BLEU: metriche complementari

BLEU confronta parola per parola contro *un solo* riferimento e non vede mai l'immagine: penalizza le riformulazioni corrette e premia la ripetizione del template. Per leggere i modelli con più onestà abbiamo affiancato a BLEU sei metriche complementari, calcolate su tutte le predizioni (4 baseline + 3 modelli, 7 attributi per-attributo e 3 versioni globali; ambiente isolato in `eval_metrics/`).

Tabella 1: Le sette metriche e cosa misurano.

Metrica	Cosa misura	Famiglia
BLEU-1/4	precisione degli n -gram (parole / fraseologia esatta)	testo
METEOR	sovrapposizione con stemming e sinonimi	testo
ROUGE-L	più lunga sottosequenza comune (ordine)	testo
CIDEr	n -gram pesati TF-IDF: premia i termini rari salienti	testo
BERTScore	similarità semantica via embedding (BERT it)	semantica
Conformità Vocab.	% di parole nel lessico sensoriale attestato	dominio
CLIPScore	appropriatezza caption-immagine (coseno CLIP)	immagine

16.1 CLIPScore conferma lo shuffle test

CLIPScore è l'unica metrica che guarda l'immagine: proietta fetta e caption nello stesso spazio CLIP multilingue e ne misura la similarità coseno, *ignorando* il riferimento del panel. Il risultato è netto e converge con lo shuffle test (Fig. 15): la media CLIPScore è **quasi identica** fra modelli addestrati e baseline costante (Fig. 16). I modelli m1/m3/m6 *non* superano *most_frequent*, che spara sempre la stessa frase ignorando l'immagine (scarto $< 0,004$, dentro il rumore). È la conferma quantitativa e indipendente che, con encoder congelato, l'immagine non viene davvero sfruttata: i modelli fanno *language modeling* sulla distribuzione delle caption.

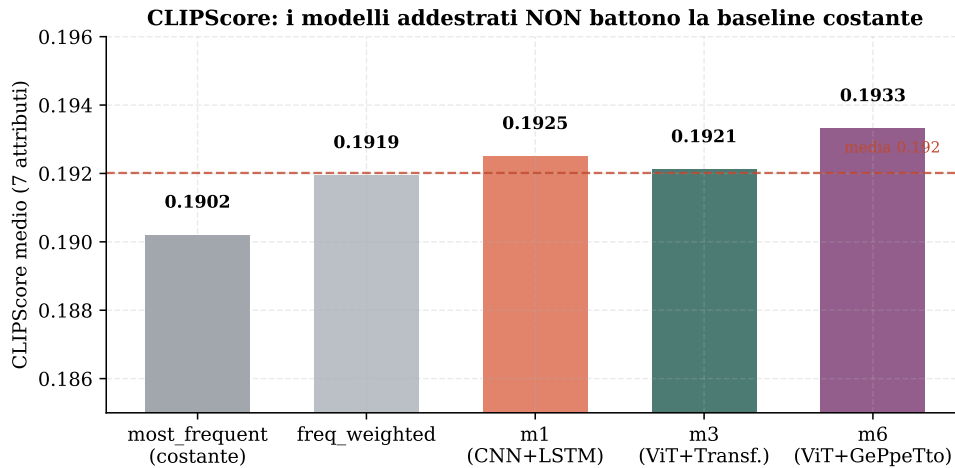


Figura 16: CLIPScore medio sui 7 attributi per-attributo. Gli scarti tra modelli addestrati e baseline costante sono dentro il rumore: nessun modello sfrutta l'immagine più di una caption fissa.

La variazione vera è **per attributo**, non per modello (Fig. 17): CLIP “vede” che texture (0,208), colore (0,206) e struttura (0,197) sono descrivibili dall'immagine, mentre aroma (0,181), sapore (0,184) e profumo (0,185) restano bassi per chiunque — nessuna immagine può rivelare un sapore. È un controllo di sanità che *valida* la metrica. Eccezione istruttiva: lo Spessore della Crosta è visibile ma basso (0,184), perché la crosta è una porzione piccola della fetta.

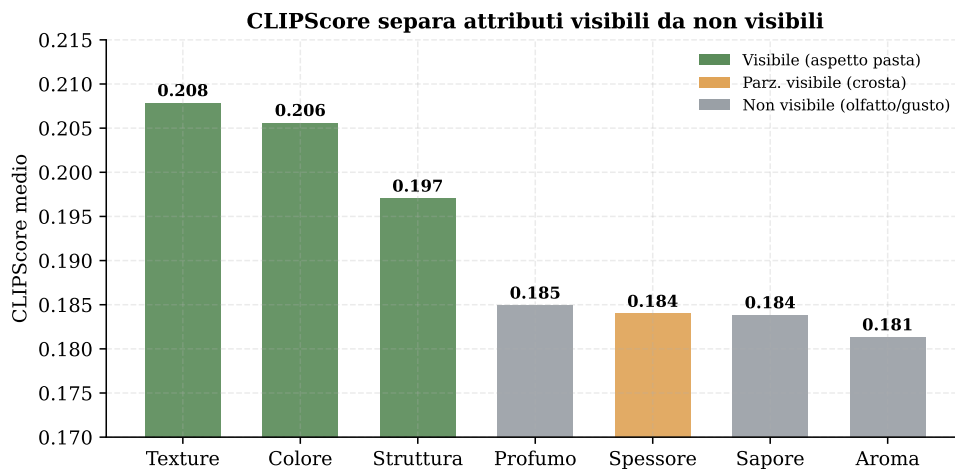


Figura 17: CLIPScore medio per attributo. Gli attributi visivi ottengono valori più alti; olfatto/gusto restano bassi. La metrica si comporta come atteso da un punteggio image-grounded.

16.2 CIDEr discrimina dove BLEU appiattisce

A differenza di BLEU-4 (valori vicini fra i modelli), CIDEr pesa i termini rari con TF-IDF e *separa* i modelli (Fig. 18). La media per-attributo li ordina correttamente (random $0,23 < \text{freq_w} < 0,37$ $m1 < m3 < m6$ $0,59 < m3 < m6 < m6$ $0,65 < m6 < m6$): i modelli addestrati battono nettamente le baseline casuali e m6 è il migliore. **most_frequent** resta in testa (0,79) solo perché il riferimento è spesso la frase frequente — di nuovo un artefatto del riferimento singolo, non qualità reale.

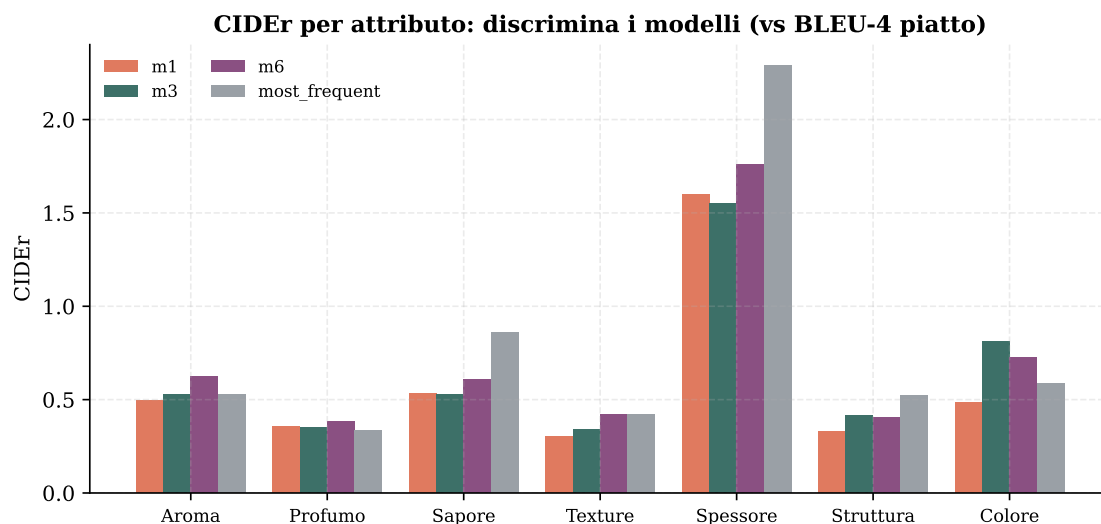


Figura 18: CIDEr per attributo. Su Spessore della Crosta, dove BLEU-4 dava 0,39–0,44 per tutti, CIDEr va da 1,04 a 2,29: un ordinamento che BLEU non vedeva.

16.3 Il tranello di BLEU, visualizzato

La Fig. 19 mostra la stessa coppia di modelli vista da due metriche: a sinistra BLEU-1, dove la caption *costante* **most_frequent** vince; a destra CLIPScore, dove i due sono praticamente uguali. Una metrica da sola può mentire — vanno lette insieme. (BERTScore, infine, resta poco discriminante qui: tutti i valori in 0,83–0,92, perché tutte le frasi parlano di formaggio con la stessa struttura; utile come *sanity check*, non per il ranking.)

Cosa aggiungono le nuove metriche

CIDEr ordina i modelli dove BLEU li appiattiva ($m1 < m3 < m6$) e premia i termini sensoriali rari. **CLIPScore** — l'unica image-grounded — mostra che i modelli addestrati non battono la baseline costante: conferma indipendente dello shuffle test (collo di bottiglia = encoder congelato), e separa gli attributi visibili da olfatto/gusto. **BERTScore** è poco discriminante su questo dominio. Il dettaglio completo è in `eval_metrics/METRICS_REPORT.md` e `metrics_summary.csv`.

17 Esempi qualitativi

I numeri raccontano metà della storia; le caption generate raccontano l'altra metà. La Tabella seguente mostra predizioni di m6 affiancate al riferimento del panel (campioni reali dal test set). Lo *scaffolding* è sempre perfetto, la lingua è italiano fluente e grammaticale; il descrittore è spesso

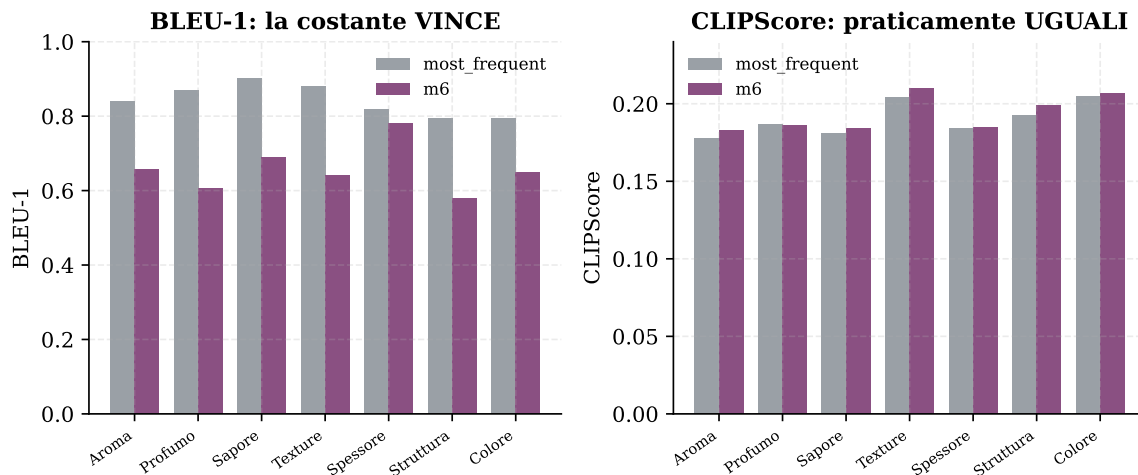


Figura 19: `most_frequent` vs `m6`. BLEU-1: la costante vince. CLIPScore: i due sono indistinguibili. BLEU-1 da solo è fuorviante.

plausibile ma riferito a un'altra dimensione sensoriale — esattamente ciò che ci si aspetta da un modello parzialmente condizionato dall'immagine che si appoggia ancora alla distribuzione marginale.

Attributo	Predizione (m6)	Riferimento (panel)
Aroma	Il formaggio ha un aroma di panna.	Il formaggio ha un aroma di panna. (<i>match esatto</i>)
Sapore	Il formaggio ha un sapore leggermente salato e piccante.	Il formaggio ha un sapore salato.
Spessore	La crosta del formaggio è mediamente spessa.	La crosta del formaggio è mediamente spessa. (<i>match</i>)
Colore	La pasta del formaggio è di colore giallo carico omogeneo.	La pasta del formaggio è di colore giallo troppo carico.
Struttura	La pasta presenta frattura irregolare e grana fine.	La pasta è stirata e poca grana a tratti.

Quattro pattern ricorrono su tutti gli attributi: (1) lo scaffolding “Il formaggio ha un...” è sempre corretto e da solo vale $\sim 50\%$ del BLEU-4; (2) la lingua è fluente grazie al LM pre-addestrato; (3) il descrittore è spesso valido ma per un *altro* formaggio; (4) la variabilità dei riferimenti tra degustatori è enorme e pone un tetto a ciò che qualsiasi modello può ottenere con BLEU a riferimento singolo.

18 Critica di BLEU e conclusioni

18.1 Perché BLEU è un cattivo giudice qui

BLEU è una metrica di puro *string-matching* e non vede mai l'immagine. Su questo dataset soffre di quattro limiti: (1) **sinonimi** senza credito (“leggero”/“lieve”/“fiavole”); (2) **equivalenza semantica** persa a livello di descrittore (“di liquirizia” vs “di anice”); (3) **riferimento singolo** (BLEU nasce multi-riferimento, ma qui c'è una sola caption del panel per immagine); (4) **gonfiaggio**

da scaffolding (il prefisso identico satura la finestra 4-gram). È proprio per questo che la baseline **most_frequent**, pur emettendo una stringa costante, supera i modelli addestrati su Aroma e Spessore: dice più sul difetto di BLEU che sulla qualità dei modelli. Per questo abbiamo calcolato metriche più adatte (§16): **CLIPScore** (appropriatezza caption-immagine), **CIDEr** (termini rari pesati TF-IDF) e la conformità al vocabolario sensoriale, oltre a METEOR.

18.2 Conclusione generale

Il progetto consegna un confronto completo di tre metodi encoder-decoder concettualmente diversi sul dataset sensoriale Trentingrana, con due risultati:

- **Risultato sostanziale (scientifico).** L’encoder visivo è il “cancello binario” dell’image-conditioning: dipende dalla scelta dell’encoder, non dal decoder né dalla scala dei dati. m1 ignora l’immagine su tutti i 7 attributi; m3 e m6 la usano su 4 su 7.
- **Risultato pratico.** I modelli addestrati battono la baseline **most_frequent** su 4 attributi su 7 (massimo: Struttura, +0,128 BLEU-4). Sui restanti pareggiano o perdono per artefatto di BLEU, non per reale fallimento (su Spessore lo shuffle test conferma che m3/m6 *usano* l’immagine).

18.3 Framing onesto

Nessuno dei modelli produce caption che sostituirebbero l’annotazione di un degustatore. Hanno appreso il *vocabolario* dei descrittori e una mappa parziale immagine→descrittore, ma non discriminano finemente all’interno delle categorie sensoriali — coerente con la scala dei dati (1.300–2.300 campioni per attributo) e con il rumore intrinseco dell’annotazione multi-degustatore.

Riepilogo Parte II

Tre metodi che spaziano gli assi encoder e decoder. m6 (ViT + GePpeTto) vince di misura quasi ovunque, ma i tre modelli sono molto vicini. Il risultato chiave dallo shuffle test: **conta l’encoder**. ResNet congelato ⇒ nessun uso dell’immagine; ViT congelato ⇒ uso dell’immagine su 4 attributi su 7, indipendentemente dal decoder. BLEU è fuorviante su questo dataset template-heavy.

18.4 Lavori futuri

In ordine di impatto atteso: (1) correggere ed eseguire la baseline di *retrieval* (k-NN su feature ResNet) come pavimento “image-aware”; (2) **fine-tuning** dell’encoder (già indicato dallo shuffle test e ora da CLIPScore come collo di bottiglia) e **ri-misura di CLIPScore**: se i modelli iniziano a staccarsi dalla baseline costante, sarebbe la prova diretta che l’immagine viene finalmente sfruttata. CLIPScore, CIDEr, BERTScore e la conformità al vocabolario sono già stati aggiunti in questa revisione (§16).